02613 Python and High Performance Computing

Answers for Week 10

---

1. GPU Reductions

This exercise is based on the excellent presentation of parallel reductions in CUDA by Mark Harris. While it's a bit dated by now, it still contains many pearls of wisdom regarding high performance CUDA code.

Consider the following implementation of a parallel reduction on a GPU:

```
from numba import cuda

TPB = 128  # Threads per block

@cuda.jit
def reduce_kernel(data, out, n):
    # Get the 1D grid and block indices
    tid = cuda.threadIdx.x
    i = cuda.grid(1)

    # Do reduction for threadblock
    s = 1
    while s < cuda.blockDim.x:
        if tid % (2 * s) == 0 and i + s < n:
            data[i] += data[i + s]
        s *= 2
        cuda.syncthreads()  # Ensure block is synchronized

    # Write result for this block to global memory
    if tid == 0:
        out[cuda.blockIdx.x] = data[i]

def get_grid(n, tpb):
    return (n + (tpb - 1)) // tpb  # Blocks per grid

def reduce(x):
    n = len(x)
    bpg = get_grid(n, TPB)
    out = cuda.device_array(bpg, dtype=x.dtype)
    while bpg > 1:
        reduce_kernel[bpg, TPB](x, out, n)
        n = bpg
        bpg = get_grid(n, TPB)
        x[:n] = out[:n]
    reduce_kernel[bpg, TPB](x, out, n)
    return out
```

1. **Autolab** Make a Python program that takes a number n as a command line argument. The program must then generate n random float32 numbers and use the above code to compute their sum. Use `np.random.rand` to generate the random numbers and convert them to float32 with `np.astype`. Finally, print the sum (and only the sum).

Answer: Autolab exercise - answer omitted

---

2. For now, do *not* manually transfer the data to the GPU - let Numba do it when the kernel is called. Run your program for $n = 4000000$ under the nsys profiler and view the statistics. Where is time spent?
   Hint: run `nsys profile -o <prof_data_file> python program.py <args>`.
   To view statistics: `nsys stats <prof_data_file>.nsys-rep`.

Answer: nsys will collect a lot of statistics about several parts of your program, and it is worth familiarizing yourself with the output. For this exercises, the relevant parts are near the bottom:

```
 1  [6/8] Executing 'gpukernsum' stats report
 2
 3  Time (%)  Total Time (ns)  Instances  Avg (ns)  Med (ns) ... Name
 4  --------  ---------------  ---------  --------  -------- ...
       ---------------------------------------------- ...
 5      91.8           204735          1  204735.0  204735.0 ...
           cudapy::__main__::reduce_kernel[abi:v1,cw51cXT ...
 6       3.2             7040          1    7040.0    7040.0 ...
           cudapy::__main__::reduce_kernel[abi:v1,cw51cXT ...
 7       2.6             5824          1    5824.0    5824.0 ...
           cudapy::__main__::reduce_kernel[abi:v1,cw51cXT ...
 8       2.4             5408          1    5408.0    5408.0 ...
           cudapy::__main__::reduce_kernel[abi:v1,cw51cXT ...
 9
10  [7/8] Executing 'gpumemtimesum' stats report
11
12  Time (%)  Total Time (ns)  Count  Avg (ns)   Med (ns)   Min (ns)
       Max (ns)  StdDev (ns)      Operation
13  --------  ---------------  -----  ---------  ---------  --------
       --------  -----------  ------------------
14      56.4         13064802      4  3266200.5  3257304.5   3239241
            3310952      31332.9  [CUDA memcpy HtoD]
15      43.6         10102040      8  1262755.0  1183783.5      1759
            2747597    1350183.9  [CUDA memcpy DtoH]
16
17  [8/8] Executing 'gpumemsizesum' stats report
18
```

```
19  Total (MB)  Count  Avg (MB)  Med (MB)  Min (MB)  Max (MB)  StdDev
       (MB)      Operation
20  ----------  -----  --------  --------  --------  --------
       -----------  ------------------
21       64.126      8     8.016     8.062     0.000    16.000
            8.536  [CUDA memcpy DtoH]
22       64.000      4    16.000    16.000    16.000    16.000
            0.000  [CUDA memcpy HtoD]
```

For the reduction, most of the time is spent in the first kernel call where (partially) reduce the full array. However, most of the time is (as expected) spent on memory transfers. We can also see that multiple memory transfers are performed - one host to device (CPU to GPU) and two device to host (GPU to CPU) for each kernel call. The reason we have two DtoH transfers is due to the `x[:n] = out[:n]` line, since `out` is a device array.

---

3. Modify your program such that your manually transfer the data to the GPU to avoid wasted transfers. Re-run your program under the nsys profiler. What has changed?

Answer: The relevant parts of the output are now:

```
 1  ** CUDA GPU Kernel Summary (gpukernsum):
 2
 3  Time (%)  Total Time (ns)  Instances  Avg (ns)  Med (ns) ... Name
 4  --------  ---------------  ---------  --------  -------- ...
       ------------------------------------------------ ...
 5       88.4           203711          1  203711.0  203711.0 ...
            cudapy::__main__::reduce_kernel[abi:v1,cw51cXT ...
 6        3.0             6913          1    6913.0    6913.0 ...
            cudapy::__main__::reduce_kernel[abi:v1,cw51cXT ...
 7        2.5             5760          2    2880.0    2880.0 ...
            cudapy::numba::cuda::cudadrv::devicearray::_as ...
 8        2.3             5280          1    5280.0    5280.0 ...
            cudapy::__main__::reduce_kernel[abi:v1,cw51cXT ...
 9        2.1             4831          1    4831.0    4831.0 ...
            cudapy::__main__::reduce_kernel[abi:v1,cw51cXT ...
10        1.8             4065          1    4065.0    4065.0 ...
            cudapy::numba::cuda::cudadrv::devicearray::_as ...
11
12  Processing [reduce_v2_gpu.sqlite] with [/appl/cuda/11.8.0/nsight-
       systems-2022.4.2/host-linux-x64/reports/...
13
14  ** GPU MemOps Summary (by Time) (gpumemtimesum):
15
16  Time (%)  Total Time (ns)  Count  Avg (ns)   Med (ns)   Min (ns)
       Max (ns)  StdDev (ns)      Operation
```

```
--------  ---------------  -----  ---------  ---------  --------
    --------  -----------  ------------------

     99.9          3347336      1  3347336.0  3347336.0   3347336
          3347336          0.0  [CUDA memcpy HtoD]

      0.1             1792      1     1792.0     1792.0      1792
               1792          0.0  [CUDA memcpy DtoH]

Processing [reduce_v2_gpu.sqlite] with [/appl/cuda/11.8.0/nsight-
    systems-2022.4.2/host-linux-x64/reports/...

** GPU MemOps Summary (by Size) (gpumemsizesum):

Total (MB)  Count  Avg (MB)  Med (MB)  Min (MB)  Max (MB)  StdDev
    (MB)      Operation
----------  -----  --------  --------  --------  --------
    -----------  ------------------

    16.000      1    16.000    16.000    16.000    16.000
        0.000  [CUDA memcpy HtoD]

     0.000      1     0.000     0.000     0.000     0.000
         0.000  [CUDA memcpy DtoH]
```

The time to run the kernels is mostly unchanged, which is expected. However, the memory transfers now take significantly less time! Also, we now only have one transfer from CPU to GPU and one transfer from GPU to CPU.

---

4. Modify the kernel such that each thread block loads its part of the data into a shared memory array. Then, do the reduction in the shared array instead of in the global `data` array. Note: this will not have a large effect on speed now, but will make the next optimization simpler (less book keeping) and faster.

Answer: Change the kernel to the following:

```
@cuda.jit
def reduce_kernel(data, out, n):
    # Declare shared memory
    sdata = cuda.shared.array(TPB, dtype=data.dtype)

    # Get the 1D grid and block indices
    tid = cuda.threadIdx.x
    i = cuda.grid(1)

    # Each thread loads one element from global to shared memory
    sdata[tid] = data[i] if i < n else 0.0
    cuda.syncthreads()  # Ensure all threads have loaded data

    # Do reduction for threadblock
```

```
15      s = 1
16      while s < cuda.blockDim.x:
17          if tid % (2 * s) == 0:
18              sdata[tid] += sdata[tid + s]
19          s *= 2
20          cuda.syncthreads()  # Ensure block is synchronized
21
22      # Write result for this block to global memory
23      if tid == 0:
24          out[cuda.blockIdx.x] = sdata[0]
```

We first declare the shared array with the same length as the number of threads in the thread block.

Notice the use of `cuda.syncthreads()` to make sure the threads in a block remain synchronized so we avoid data races. Also, notice we now use `tid` instead of `i` since we are now indexing into the shared array instead of global memory.

Looking at the profiling output, we indeed see that the kernel is slightly faster but not a lot.

```
 1  ** CUDA GPU Kernel Summary (gpukernsum):
 2
 3  Time (%)  Total Time (ns)  Instances  Avg (ns)  Med (ns) ... Name
 4  --------  ---------------  ---------  --------  -------- ...
       ---------------------------------------------- ...
 5      87.7           188670          1  188670.0  188670.0 ...
           cudapy::__main__::reduce_kernel[abi:v1,cw51cXT ...
 6       3.1             6624          1    6624.0    6624.0 ...
           cudapy::__main__::reduce_kernel[abi:v1,cw51cXT ...
 7       2.7             5729          2    2864.5    2864.5 ...
           cudapy::numba::cuda::cudadrv::devicearray::_as ...
 8       2.3             4960          1    4960.0    4960.0 ...
           cudapy::__main__::reduce_kernel[abi:v1,cw51cXT ...
 9       2.3             4960          1    4960.0    4960.0 ...
           cudapy::__main__::reduce_kernel[abi:v1,cw51cXT ...
10       2.0             4256          1    4256.0    4256.0 ...
           cudapy::numba::cuda::cudadrv::devicearray::_as ...
11
12  Processing [gpureduce_v3_gpu.sqlite] with [/appl/cuda/11.8.0/
       nsight-systems-2022.4.2/host-linux-x64/reports/...
13
14  ** GPU MemOps Summary (by Time) (gpumemtimesum):
15
16  Time (%)  Total Time (ns)  Count  Avg (ns)   Med (ns)   Min (ns)
       Max (ns)  StdDev (ns)      Operation
17  --------  ---------------  -----  ---------  ---------  --------
       --------  -----------  ------------------
18      99.9          3356778      1  3356778.0  3356778.0   3356778
            3356778          0.0  [CUDA memcpy HtoD]
19       0.1             1792      1     1792.0     1792.0      1792
```

```
                1792          0.0  [CUDA memcpy DtoH]

Processing [gpureduce_v3_gpu.sqlite] with [/appl/cuda/11.8.0/
    nsight-systems-2022.4.2/host-linux-x64/reports/....

** GPU MemOps Summary (by Size) (gpumemsizesum):

Total (MB)  Count  Avg (MB)  Med (MB)  Min (MB)  Max (MB)  StdDev
    (MB)      Operation
----------  -----  --------  --------  --------  --------
    -----------  ------------------
    16.000      1    16.000    16.000    16.000    16.000
        0.000  [CUDA memcpy HtoD]
     0.000      1     0.000     0.000     0.000     0.000
         0.000  [CUDA memcpy DtoH]
```

5. A major reason the current kernel is slow is the `if tid % (s * 2)== 0` condition. This causes a lot of warp divergence which slows the kernel down. Instead of "masking out" threads, change the kernel to use a strided index as shown on slide 11 in the linked presentation by Mark Harris.

Answer: Change the kernel to:

```
@cuda.jit
def reduce_kernel(data, out, n):
    # Declare shared memory
    sdata = cuda.shared.array(TPB, dtype=data.dtype)

    # Get the 1D grid and block indices
    tid = cuda.threadIdx.x
    i = cuda.grid(1)

    # Each thread loads one element from global to shared memory
    sdata[tid] = data[i] if i < n else 0.0
    cuda.syncthreads()  # Ensure all threads have loaded data

    # Do reduction for threadblock
    s = 1
    while s < cuda.blockDim.x:
        idx = 2 * s * tid

        if idx < cuda.blockDim.x:
            sdata[idx] += sdata[idx + s]
        s *= 2
        cuda.syncthreads()  # Ensure block is synchronized

    # Write result for this block to global memory
    if tid == 0:
```

```
26          out[cuda.blockIdx.x] = sdata[0]
```

Instead of the masking, we now compute a strided index as `idx = 2 * s * tid`. This decreases the warp divergence, as threads within a warp more often perform the same operation.

6. Profile the optimized kernel. What has changed?

Answer: The profiling results are now:

```
 1  ** CUDA GPU Kernel Summary (gpukernsum):
 2
 3  Time (%)  Total Time (ns)  Instances  Avg (ns)  Med (ns) ... Name
 4  --------  ---------------  ---------  --------  -------- ...
       ---------------------------------------------- ...
 5      78.6            81696          1   81696.0   81696.0 ...
           cudapy::__main__::reduce_kernel[abi:v1,cw51cXT ...
 6       5.6             5792          2    2896.0    2896.0 ...
           cudapy::numba::cuda::cudadrv::devicearray::_as ...
 7       4.8             4991          1    4991.0    4991.0 ...
           cudapy::__main__::reduce_kernel[abi:v1,cw51cXT ...
 8       4.0             4160          1    4160.0    4160.0 ...
           cudapy::numba::cuda::cudadrv::devicearray::_as ...
 9       3.5             3680          1    3680.0    3680.0 ...
           cudapy::__main__::reduce_kernel[abi:v1,cw51cXT ...
10       3.5             3648          1    3648.0    3648.0 ...
           cudapy::__main__::reduce_kernel[abi:v1,cw51cXT ...
11
12  Processing [gpureduce_v4_gpu.sqlite] with [/appl/cuda/11.8.0/
       nsight-systems-2022.4.2/host-linux-x64/reports/...
13
14  ** GPU MemOps Summary (by Time) (gpumemtimesum):
15
16  Time (%)  Total Time (ns)  Count  Avg (ns)   Med (ns)   Min (ns)
       Max (ns)  StdDev (ns)      Operation
17  --------  ---------------  -----  ---------  ---------  --------
       --------  -----------  ------------------
18      99.9          3279721      1  3279721.0  3279721.0   3279721
            3279721          0.0  [CUDA memcpy HtoD]
19       0.1             1792      1     1792.0     1792.0      1792
                1792          0.0  [CUDA memcpy DtoH]
20
21  Processing [gpureduce_v4_gpu.sqlite] with [/appl/cuda/11.8.0/
       nsight-systems-2022.4.2/host-linux-x64/reports/...
22
23  ** GPU MemOps Summary (by Size) (gpumemsizesum):
24
25  Total (MB)  Count  Avg (MB)  Med (MB)  Min (MB)  Max (MB)  StdDev
       (MB)      Operation
```

```
26  ----------  -----  --------  --------  --------  --------
        -----------  ------------------
27       16.000      1    16.000    16.000    16.000    16.000
            0.000  [CUDA memcpy HtoD]
28        0.000      1     0.000     0.000     0.000     0.000
             0.000  [CUDA memcpy DtoH]
```

The time to run the kernel has now decreased from 188670 ns to 81696 ns, about 2.3x faster!

---

7. (Optional) These exercises were only the first few optimizations presented in the above reference. There is still plenty of performance to be squeezed out! Implement some (or all) of the remaining optimizations. The code in the presentation is in C++, but everything translates neatly to Numba functions.

   Note also, that modern CUDA versions have introduced helper functions that can now make reductions even faster. For more info on this, see this technical blog from NVIDIA.

   Hint: you may need to apply the reduction to a larger set of numbers to really see speed benefits.

Answer: Applying all the optimization results in the following code:

```
 1  TPB = 126  # Threads per block
 2
 3  @cuda.jit(device=True)
 4  def warp_reduce(sdata, tid):
 5      sdata[tid] += sdata[tid + 32]
 6      sdata[tid] += sdata[tid + 16]
 7      sdata[tid] += sdata[tid + 8]
 8      sdata[tid] += sdata[tid + 4]
 9      sdata[tid] += sdata[tid + 2]
10      sdata[tid] += sdata[tid + 1]
11
12  @cuda.jit
13  def reduce_kernel(data, out, n):
14      # Declare shared array
15      sdata = cuda.shared.array(shape=TPB, dtype=data.dtype)
16
17      # Each thread loads many elements from global to shared memory
18      tid = cuda.threadIdx.x
19      i = cuda.blockIdx.x * cuda.blockDim.x * 2 + cuda.threadIdx.x
20      grid_size = cuda.blockDim.x * 2 * cuda.gridDim.x
21
22      sdata[tid] = 0.0
23      while i < n:
24          sdata[tid] += data[i]
25          sdata[tid] += data[i + cuda.blockDim.x] \
26                        if i + cuda.blockDim.x < n else 0.0
```

```
27          i += grid_size
28      cuda.syncthreads()
29
30      # Do reduction in shared memory
31      if TPB >= 512:
32          if tid < 256:
33              sdata[tid] += sdata[tid + 256]
34          cuda.syncthreads()
35      if TPB >= 256:
36          if tid < 128:
37              sdata[tid] += sdata[tid + 128]
38          cuda.syncthreads()
39      if TPB >= 128:
40          if tid < 64:
41              sdata[tid] += sdata[tid + 64]
42          cuda.syncthreads()
43
44      if tid < 32:
45          warp_reduce(sdata, tid)
46
47      # Write result for this block to global memory
48      if tid == 0:
49          out[cuda.blockIdx.x] = sdata[0]
50
51  def get_grid(n, tpb):
52      return (n + (tpb - 1)) // tpb  # Blocks per grid
53
54  def reduce(x):
55      n = len(x)
56      bpg = get_grid(n, TPB)
57      bpg = (bpg + 1) // (2 * 2048)  # Determined by playing with
            values
58      out = cuda.device_array(bpg, dtype=x.dtype)
59      while bpg > 1:
60          reduce_kernel[bpg, TPB](x, out, n)
61          n = bpg
62          bpg = get_grid(n, TPB)
63          bpg = (bpg + 1) // 2
64          x[:n] = out[:n]
65      reduce_kernel[bpg, TPB](x, out, n)
66      return out
```

To really see the benefits of the extra optmizations, we apply it to 400,000,000 random numbers. The strided index version takes 7.61 ms to run all kernels (98% of the time is spent in the first kernel launch). The version with all optimizations takes 1.85 ms. Roughly a 4.1x speed up. Adding up the speed ups, the fully optimized kernel is roughly 9.4x faster. Almost an order of magnitude!

2. CuPy

In this exercise, we will use CuPy to speed up existing NumPy code - specifically, the haversine distance code we worked with in week 4. The single loop and no loop distance matrix implementations were:

```
def distance_matrix_oneloop(p1, p2):
    p1 = np.radians(p1)
    p2 = np.radians(p2)

    D = np.empty((len(p1), len(p2)))
    for i in range(len(p1)):
        dsin2 = np.sin(0.5 * (p1[i] - p2)) ** 2
        cosprod = np.cos(p1[i, 0]) * np.cos(p2[:, 0])
        a = dsin2[:, 0] + cosprod * dsin2[:, 1]
        row = 2 * np.arctan2(np.sqrt(a), np.sqrt(1 - a))
        D[i, :] = row

    D *= 6371  # Earth radius in km
    return D

def distance_matrix_noloop(p1, p2):
    p1 = np.radians(p1)
    p2 = np.radians(p2)
    dsin2 = np.sin(0.5 * (p1[:, None, :] - p2[None, :, :])) ** 2
    cosprod = np.cos(p1[:, None, 0]) * np.cos(p2[None, :, 0])
    D = 2 * np.arcsin(np.sqrt(dsin2[:, :, 0] + cosprod * dsin2[:, :,
       1]))
    D *= 6371  # Earth radius in km
    return D
```

For reference, the location data was stored at:
`/dtu/projects/02613_2025/data/locations/`

1. **Autolab** Convert both implementations such that they use CuPy operations instead of NumPy. Also modify your Python program from week 4 that used the distance matrix functions to use CuPy.

   Answer: Autolab exercise - functions omitted.

   The modified python program:

   ```
   import sys
   import cupy as cp

   def distance_matrix(p1, p2):
       ...

   def load_points(fname):
   ```

```
 8      data = cp.loadtxt(fname, delimiter=',', skiprows=1, usecols
           =(1, 2))
 9      return data
10
11  def distance_stats(D):
12      # Extract upper triangular part to avoid duplicate entries
13      assert D.shape[0] == D.shape[1], 'D must be square'
14      idx = cp.triu_indices(D.shape[0], k=1)
15      distances = D[idx]
16      return {
17          'mean': float(distances.mean()),
18          'std': float(distances.std()),
19          'max': float(distances.max()),
20          'min': float(distances.min()),
21      }
22
23  fname = sys.argv[1]
24  d_points = load_points(fname)
25  D = distance_matrix_v4(d_points, d_points)
26  stats = distance_stats(D)
27  print(stats)
```

2. Measure the run time for your CuPy program using the single loop version. Use the subset with 5000 rows `locations_5000.csv` file. How does the run time compare to the CPU version from week 4? You may need to run the programs a few times to get stable times. For simplicity, you may simply measure the run time with the `time` command: `time python points.py <path to data>`

Answer: The exact run time will depend on your hardware. The NumPy, using the fastest CPU implementation of `distance_matrix`, took 10.42 s to run. The single loop CuPy version took 3.29 s. Over 3x speedup for very little work!

3. Re-run the program under the nsys profiler. Where does the program spend its time? Be sure to check all sections (except the OS Runtime Summary) to get a full overview.

Answer: The relevant sections from the profiler output are:

```
1  ** CUDA API Summary (cudaapisum):
2
3  Time (%)  Total Time (ns)  Num Calls  Avg (ns)   Med (ns)  Min (ns
       )  Max (ns)   StdDev (ns)              Name
```

```
 4  --------  ---------------  ---------  ---------  --------
       --------  ---------  -----------  ------------------...
 5      65.0         619355626     135485     4571.4    4529.0
           3825      61385        656.1  cuLaunchKernel
 6      12.8         121792084         19  6410109.7   29574.0
           4338  119486636   27383206.4  cudaMalloc
 7      12.6         119554601      13552     8821.9    7941.0
           7368    6272166      58839.1  cudaMemcpyAsync
 8       9.4          89442587         28  3194378.1  310908.5
            112381   68891590   12923772.9  cuModuleLoadData
 9       0.1           1063756         16    66484.8   64800.5
            47532      92025      13055.4  cuModuleUnload
10  ...
11
12  ** CUDA GPU Kernel Summary (gpukernsum):
13
14  Time (%)  Total Time (ns)  Instances   Avg (ns)    Med (ns)   Min
       (ns)  Max (ns) ... Name
15  --------  ---------------  ---------  ----------  ----------
       --------  -------- ... -------------------------...
16      15.0          84552752      27090      3121.2      3104.0
           3071     14176 ... cupy_multiply__float64_fl...
17      12.4          69762607          1  69762607.0  69762607.0
           69762607  69762607 ... cupy_var_core_float64
18      10.3          58060436      13545      4286.5      4288.0
           4223     14207 ... cupy_power__float64_float...
19       8.7          48786568      13545      3601.8      3584.0
            3551     13760 ... cupy_sin__float64_float64
20       8.6          48322893      13545      3567.6      3552.0
            3487     14687 ... cupy_subtract__float64_fl...
21       8.3          46863503      13545      3459.8      3456.0
            3360     14591 ... cupy_arcsin__float64_floa...
22       7.7          43344898      13545      3200.1      3200.0
            3135      9920 ... cupy_add__float64_float64...
23       7.6          43024135      13545      3176.4      3168.0
            3135      3552 ... cupy_multiply__float_floa...
24       7.5          42030229      13545      3103.0      3072.0
            3039     14112 ... cupy_sqrt__float64_float6...
25       7.2          40577091      13545      2995.7      2976.0
            2943      8864 ... cupy_multiply__float_floa...
26       1.1           6179516          2   3089758.0   3089758.0
            3086974   3092542 ... cupy_getitem_mask
27       0.9           5320645          2   2660322.5   2660322.5
            2659490   2661155 ... cupy_scan_naive
28  ...
29
30  ** GPU MemOps Summary (by Time) (gpumemtimesum):
31
32  Time (%)  Total Time (ns)  Count  Avg (ns)  Med (ns)  Min (ns)
       Max (ns)  StdDev (ns)      Operation
33  --------  ---------------  -----  --------  --------  --------
```

```
        --------  -----------  ------------------
34       97.8          38741966  13545    2860.2    2848.0      2815
                14848        114.4  [CUDA memcpy DtoD]
35        2.1           818870      1  818870.0  818870.0    818870
             818870          0.0  [CUDA memset]
36        0.1            21248      1   21248.0   21248.0     21248
                 21248          0.0  [CUDA memcpy HtoD]
37        0.0            12770      6    2128.3    2048.0      1312
                  3073        795.4  [CUDA memcpy DtoH]
38  ...
39  ** GPU MemOps Summary (by Size) (gpumemsizesum):
40
41  Total (MB)  Count  Avg (MB)  Med (MB)  Min (MB)  Max (MB)  StdDev
       (MB)      Operation
42  ----------  -----  --------  --------  --------  --------
       -----------  ------------------
43  1467.736  13545     0.108     0.108     0.108     0.108
       0.000  [CUDA memcpy DtoD]
44   733.814      1   733.814   733.814   733.814   733.814
        0.000  [CUDA memset]
45     0.217      1     0.217     0.217     0.217     0.217
          0.000  [CUDA memcpy HtoD]
46     0.000      6     0.000     0.000     0.000     0.000
          0.000  [CUDA memcpy DtoH]
```

Notice that a \emph{significant} part of the run time is spend on launching kernels (the `cuLaunchKernel` line in the `CUDA API Summary`) and on moving data around on the GPU (the `[CUDA memcpy DtoD]` line in the `GPU MemOps Summary`). In fact, the `cuLaunchKernel` calls take more time than the sum of all the kernel run times!

This tells us, that for CuPy, repeatedly performing smaller operations is likely not a good idea. This is different from NumPy where the single loop version did not suffer significant performance issues.

---

4. Measure the run time for your CuPy program using the no loop version. What is the new run time?

Answer: Again, the exact run time will depend on your hardware. the no loop CuPy version took 1.00 s. Again, over 3x speed-up compared the previous CuPy version and an order of magnitude faster than the NumPy version!

---

5. Re-run the program under the nsys profiler. Where does the program now spend its time? What has changed compared to the single loop version? Be sure to check all sections to get a full

overview.

Answer: The relevant sections from the profiler output are

```
** CUDA API Summary (cudaapisum):

Time (%)  Total Time (ns)  Num Calls   Avg (ns)   Med (ns)   Min (
    ns)  Max (ns)   StdDev (ns)  Name
--------  ---------------  ---------  ----------  ---------
    --------  ---------  -----------  ---------------...
     53.0        134316747          29   4631612.0  1724601.0
        110696   65353278   11994929.0  cuModuleLoadData
     41.8        105965618           7  15137945.4   211721.0
        7091  104289102   39313330.6  cudaMalloc
      3.9         10016516           7   1430930.9   783843.0
        19298    5940196    2164289.5  cudaMemcpyAsync
      0.6          1471680          16     91980.0    73759.5
        45619     303437      61285.0  cuModuleUnload
      0.3           856393           1    856393.0   856393.0
        856393     856393           0.0  cudaHostAlloc
      0.2           557596          45     12391.0    10535.0
        4582      32132       7344.1  cuLaunchKernel
      0.1           185209           8     23151.1    18951.0
        6243      45549      16293.8  cudaLaunchKernel
      0.0            93177         384       242.6      202.0
        117        919         111.6  cuGetProcAddress
      0.0            42672           1     42672.0    42672.0
        42672      42672           0.0  cudaMemGetInfo
      0.0            20659           6      3443.2     3177.0
        2296       5448       1204.4  cudaStreamSynch...
      0.0            14990           7      2141.4     1576.0
        779        6026       1835.7  cudaStreamIsCap...
      0.0            14421           1     14421.0    14421.0
        14421      14421           0.0  cudaMemsetAsync
      0.0            12210           1     12210.0    12210.0
        12210      12210           0.0  cudaEventCreate...
      0.0             6565           1      6565.0     6565.0
        6565       6565          0.0  cudaEventRecord
      0.0             4016           1      4016.0     4016.0
        4016       4016          0.0  cudaEventQuery
      0.0             1970           2       985.0      985.0
        155        1815       1173.8  cuModuleGetLoad...
      0.0             1670           1      1670.0     1670.0
        1670       1670          0.0  cudaEventDestro...
      0.0             1095           1      1095.0     1095.0
        1095       1095          0.0  cuInit

** CUDA GPU Kernel Summary (gpukernsum):

Time (%)  Total Time (ns)  Instances   Avg (ns)    Med (ns)   Min
```

```
       (ns)  Max (ns) ... Name
 --------  ---------------  ---------  ----------  ----------
       --------  -------- ... -------------------------...
      40.6         66124313          1  66124313.0  66124313.0
          66124313  66124313 ... cupy_var_core_float64
       7.6         12433525          1  12433525.0  12433525.0
          12433525  12433525 ... cupy_power__float64_float...
       5.8          9468854          2   4734427.0   4734427.0
          2484740   6984114 ... cupy_multiply__float64_fl...
       4.7          7620778          1   7620778.0   7620778.0
          7620778   7620778 ... cupy_sin__float64_float64
       4.5          7255695          1   7255695.0   7255695.0
          7255695   7255695 ... cupy_multiply__float_floa...
       4.3          6965714          1   6965714.0   6965714.0
          6965714   6965714 ... cupy_add__float64_float64...
       3.7          5979134          2   2989567.0   2989567.0
          2986847   2992287 ... cupy_getitem_mask
       3.2          5271684          2   2635842.0   2635842.0
          2635554   2636130 ... cupy_prepare_array_indexi...
       3.0          4964808          2   2482404.0   2482404.0
          2481732   2483076 ... cupy_scan_naive
       3.0          4964072          1   4964072.0   4964072.0
          4964072   4964072 ... cupy_subtract__float64_fl...
       3.0          4953032          2   2476516.0   2476516.0
          2471140   2481892 ... cupy_copy__int64_int64
       2.2          3659543          1   3659543.0   3659543.0
          3659543   3659543 ... cupy_arcsin__float64_floa...
       2.2          3630007          1   3630007.0   3630007.0
          3630007   3630007 ... cupy_sqrt__float64_float6...
       2.2          3624407          1   3624407.0   3624407.0
          3624407   3624407 ... cupy_multiply__float_floa...
       2.2          3622423          1   3622423.0   3622423.0
          3622423   3622423 ... cupy_multiply__float64_fl...
       1.6          2648898          1   2648898.0   2648898.0
          2648898   2648898 ... cupy_take
       1.5          2472997          1   2472997.0   2472997.0
          2472997   2472997 ... cupy_tri
       1.5          2471844          1   2471844.0   2471844.0
          2471844   2471844 ... cupy_invert__bool_bool
       1.0          1660398          2    830199.0    830199.0
          826967    833431 ... void cub::CUB_200200_350_...
       0.8          1383153          2    691576.5    691576.5
          664601    718552 ... cupy_bsum_shfl
       0.5           833207          1    833207.0    833207.0
          833207    833207 ... void cub::CUB_200200_350_...
 ...

 ** GPU MemOps Summary (by Time) (gpumemtimesum):

 Time (%)  Total Time (ns)  Count  Avg (ns)  Med (ns)  Min (ns)
       Max (ns)  StdDev (ns)      Operation
```

```
54  --------  ---------------  -----  --------  --------  --------
       --------  -----------  ------------------
55      96.3           818806      1  818806.0  818806.0    818806
           818806          0.0  [CUDA memset]
56       2.5            21312      1   21312.0   21312.0     21312
                 21312          0.0  [CUDA memcpy HtoD]
57       1.2            10401      6    1733.5    1472.5      1280
                  3040        668.9  [CUDA memcpy DtoH]
58
59  ...
60
61  ** GPU MemOps Summary (by Size) (gpumemsizesum):
62
63  Total (MB)  Count  Avg (MB)  Med (MB)  Min (MB)  Max (MB)  StdDev
       (MB)      Operation
64  ----------  -----  --------  --------  --------  --------
       -----------  ------------------
65      733.814      1   733.814   733.814   733.814   733.814
           0.000  [CUDA memset]
66        0.217      1     0.217     0.217     0.217     0.217
              0.000  [CUDA memcpy HtoD]
67        0.000      6     0.000     0.000     0.000     0.000
              0.000  [CUDA memcpy DtoH]
```

Now, the overhead from `cuLaunchKernel` is gone. We can also see that each kernel is only called 1-2 times instead of thounsands (the `Instances`) column under `CUDA GPU Kernel Summary`. Finally, we also see that we are no longer moving data around on the GPU as the `[CUDA memcpy DtoD]` line is now gone.

Moral: when using CuPy it is extra important to make good use of large array operations instead of many small ones, in order to really take full advantage of the GPU.